

How to read the software

fe-large-displacement

Abaqus is several programs that share a name. This repository is a thin layer on top of them. Confusing the parts is how a three-day job is spent on a keyword CAE never wrote.

1 Abaqus/CAE versus the solvers

Abaqus/CAE builds a model. It does not solve it. Two solvers do:

- Abaqus/Standard — implicit. Equilibrium iterations each increment.
- Abaqus/Explicit — explicit dynamics. A stable time increment, no iteration.

CAE writes an input file (`.inp`). The solver reads that file. CAE accepts arguments it does not implement and omits keywords whose values match the default. The deck is the ground truth. Every model here writes the `.inp`, checks it, and only then submits.

2 Standard versus Explicit

	Standard	Explicit
Procedure	*Static (here)	*Dynamic, Explicit
Failure mode	no convergence	element distortion
User material	UMAT	VUMAT
Pore-pressure elements	C3D8P	none on ordinary solids
ALE	limited	production tool
CEL	none	Explicit only
Precision	double	set <code>DOUBLE_PLUS_PACK</code>

Standard is the right tool until the soil starts to fail in a large-displacement sense. Explicit is the right tool after that, provided the run is shown to be quasi-static (`ALLKE/ALLIE`). See `docs/01-explicit-quasi-static.md`.

UMAT and VUMAT are different calling conventions for the same constitutive idea. A closed-form Mohr–Coulomb return can be wrapped. A model with substepping cannot be wrapped blindly. Verify on one element (`constitutive/single_element`) before a penetration job.

3 ALE and CEL are solver features

They are not separate products.

ALE keeps the mesh topology and moves nodes relative to the material. Explicit ALE is what this repository uses in production. Standard ALE exists on geometrically nonlinear ***Static** steps. It has one smoothing algorithm, no initial mesh sweeps, and Lagrangian domains only. The implicit-ALE model shows that path. It is not the tool for skirt penetration.

CEL lets material flow through a fixed Eulerian mesh. The structure stays Lagrangian. CEL is Explicit only. There is no implicit CEL.

4 What this repository is

<code>lib/fldlib/</code>	thin wrappers over one API call or one idiom each
<code>lib/postproc/</code>	ODB energy and history extraction
<code>examples/indenter/</code>	one strip-indenter problem, five formulations
<code>examples/suction_caisson/</code>	introduction to caisson installation
<code>examples/common/</code>	shared indenter builder and run loop
<code>constitutive/</code>	UMAT/VUMAT sources, each with its own licence
<code>docs/</code>	measured findings from Abaqus 2021 jobs that were actually run
<code>tools/</code>	<code>check_odb_finite.py</code> and similar
<code>teaching/</code>	this PDF, and the formulations companion

`fldlib` does not hide state and does not skip work silently. If a helper cannot do what it was asked, it raises.

5 How a job is launched

```
set FLD_WORKDIR=C:\abq\run
set FLD_SUBMIT=0
abaqus cae noGUI=examples/indenter/implicit/model.py
```

`FLD_SUBMIT=0` builds and checks decks without solving. Start there. `FLD_CASES`, `FLD_NCPU` and `FLD_<PARAM>` override declared parameters. The log prints the full parameter table before anything is built.

When a user subroutine is compiled, launch through `abaqus.bat`, never `ABQLauncher.exe` directly. Paths with spaces break the Fortran compile. See `docs/abaqus-launch-guide.md`.

6 The habit that catches silent failures

Write the deck. Read it. Assert. Then submit. Three failures produce a normal ODB and no warning:

1. An ALE domain that is defined but never sweeps — `fldlib.ale.assert_active` reads the `.msg`.
2. A CEL domain with no void for heave — `fldlib.cel.check_void_fraction`.
3. A VUMAT returning NaN — Explicit integrates it to the end. `tools/check_odb_finite.py`.

7 What is measured

The findings in `docs/` came from Abaqus 2021 jobs that were run. Trust those numbers. On a new machine, write and check the decks with `FLD_SUBMIT=0` before submitting.

The code that changes from implicit to CEL is `teaching/formulations.pdf`.